

AgentFlow

Plateforme de création et de déploiement d'agents RAG

Étudiant : Fadi Khala

Projet : AgentFlow

Encadrant : Pr. Omri

Date : 13 juillet 2026

1. Introduction et contexte

Ce document présente l'état d'avancement du projet AgentFlow, une plateforme web permettant à une organisation de construire, configurer, déployer et exploiter des agents conversationnels basés sur la technologie RAG (Retrieval-Augmented Generation), à partir de ses propres documents internes et sans écrire de code.

L'idée centrale est de séparer clairement trois rôles au sein d'une même organisation :

- l'Administrateur, qui gère l'organisation, les départements, les utilisateurs et les fournisseurs de modèles (clés API) ;
- l'ingénieur IT, qui construit et règle finement les pipelines RAG, les teste, puis les déploie ;
- l'utilisateur final (end-user), qui interroge en langage naturel les agents mis à sa disposition et consulte les documents sources.

La plateforme est multi-organisation et cloisonnée par département (ex. RH, Finance, Juridique), chaque département disposant de ses propres espaces documentaires et de ses propres agents.

2. Objectifs du projet

Objectif général. Offrir un outil « no-code » complet qui rend accessible la mise en place d'un système RAG de qualité professionnelle, tout en laissant à l'ingénieur IT le contrôle fin de chaque étape du pipeline.

Objectifs spécifiques

- Prendre en charge de multiples sources et formats de documents (PDF, DOCX, PPTX, CSV, XLSX, HTML, pages web, Google Drive).
- Rendre chaque étape du pipeline configurable : découpage (chunking), vectorisation (embedding), recherche et génération.
- Permettre l'itération : tester plusieurs configurations, les sauvegarder en versions, puis déployer la meilleure.
- Gérer finement les permissions (propriétaire, collaborateurs IT, accès des utilisateurs finaux) et la confidentialité des espaces.
- Fournir une expérience utilisateur soignée pour la consultation des agents et des documents.

3. Méthodologie et organisation du travail

Le développement suit une démarche itérative et incrémentale, organisée par lots fonctionnels. Chaque itération livre une fonctionnalité complète et testée de bout en bout : conception, implémentation côté backend (services et modèles), implémentation côté frontend (React), puis vérification sur une base de données réelle.

- Versionnement du code avec Git et suivi continu des évolutions.
- Migrations de schéma légères et idempotentes, exécutées au démarrage, pour faire évoluer la base sans rupture ni outil externe.
- Tests de fumée systématiques à chaque étape (permissions, cycle de déploiement, accès) sur la base PostgreSQL du projet.

- Principe directeur : expérience « no-code » pour l'utilisateur final, contrôle complet pour l'ingénieur IT.

4. Architecture technique

Le projet suit une architecture en couches, avec une séparation nette entre les routes HTTP, la logique métier (services) et les modèles de données.

Couche	Technologies et rôle
Frontend	React + Vite — interfaces distinctes par rôle (Admin, IT, End-user), design responsive.
Backend	FastAPI (Python) — API REST, authentification par jeton, contrôle d'accès par rôle.
Base de données	PostgreSQL + extension pgvector pour le stockage et la recherche vectorielle.
Fournisseurs	Groq, OpenAI, Ollama (LLM), BGE-M3 / Cohere (embeddings), Docling (parsing), Crawl4AI (web), Google Drive.

La couche « fournisseurs » est entièrement modulaire : chaque famille (chargeurs, parseurs, nettoyeurs, découpage, embedding, LLM) est isolée derrière une factory qui résout dynamiquement le fournisseur et le modèle à partir de la configuration de l'espace.

5. Modèle de données

Les principales entités de la base reflètent directement le domaine métier :

Entité	Rôle
Organisation / Département / Utilisateur	Structure multi-organisation et rôles (Admin, IT, End-user).
RAGSpace	Espace RAG : configuration complète du pipeline, statut, propriétaire et visibilité.
RAGSpaceVersion	Instantané (snapshot) d'une configuration ; support du versioning et du déploiement.
RAGSpaceCollaborator	IT co-constructeurs autorisés par le propriétaire.
RAGSpaceAccess	Liste des utilisateurs finaux autorisés à interroger un espace publié.
Document	Fichier source et son cycle de vie : UPLOADING -> LOADED -> EXTRACTED -> INDEXED.
Chunk	Fragment de texte vectorisé, stocké dans pgvector avec ses métadonnées.

6. Conception

Cette section illustre, à l'aide de diagrammes de séquence, les principaux scénarios d'interaction entre l'utilisateur, l'interface, l'API, la base de données et les modèles.

Création d'un espace RAG

L'ingénieur IT renseigne la configuration ; l'API vérifie ses droits, crée l'espace (propriétaire, visibilité, statut Brouillon) et enregistre les règles d'accès.

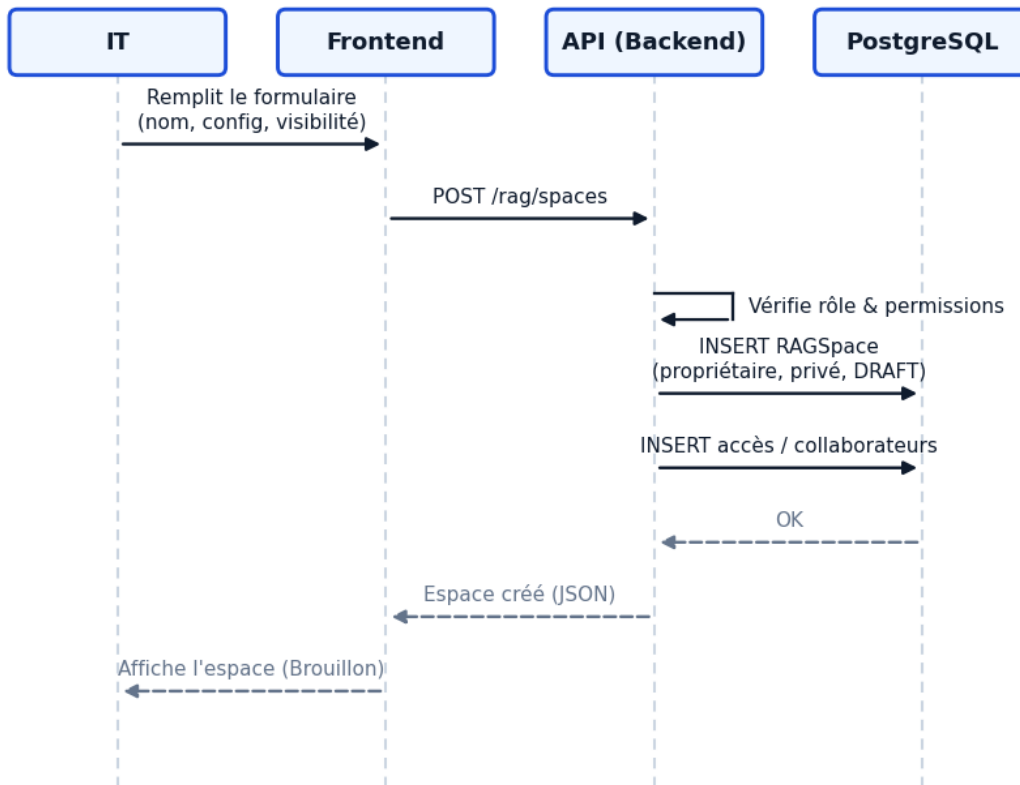


Diagramme de séquence — Création d'un espace RAG

Exécution du système RAG

À chaque question, l'API contrôle l'accès, vectorise la requête, effectue une recherche hybride dans pgvector, puis fait générer la réponse par le LLM avec les passages retrouvés comme contexte.

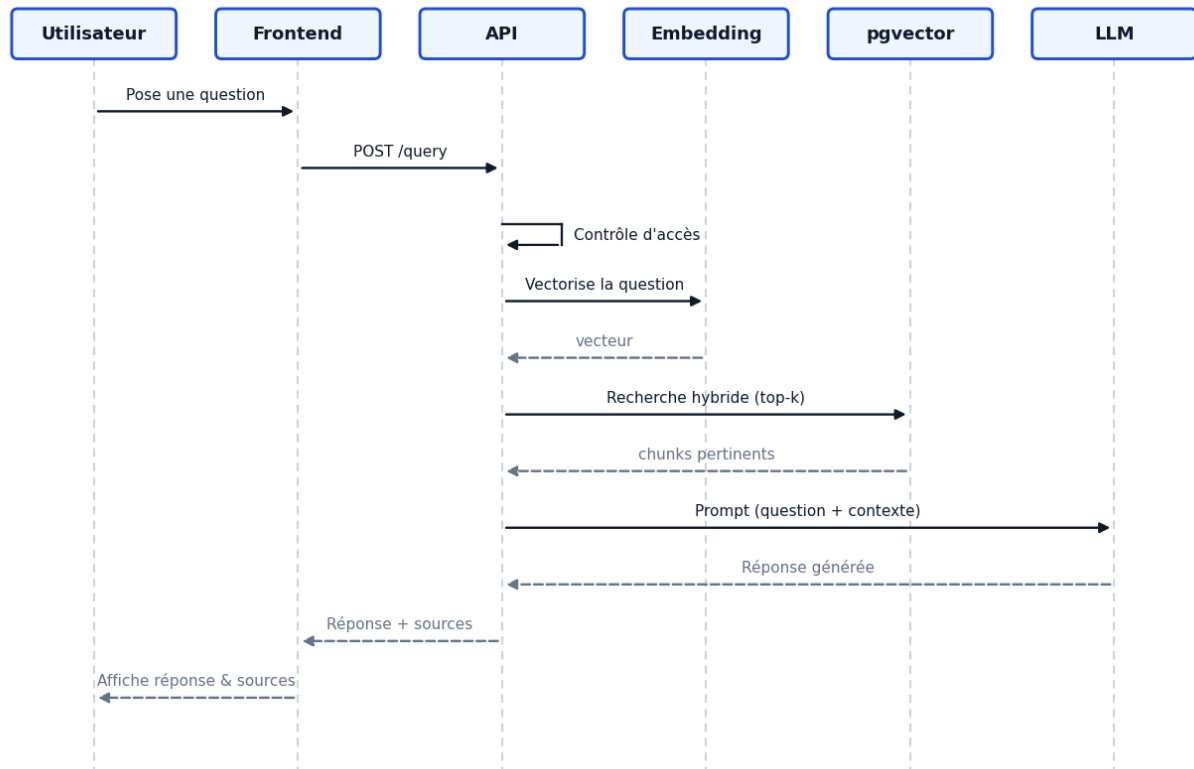


Diagramme de séquence — Exécution du système RAG

Gestion des clés API

Les clés fournisseurs ne sont jamais stockées en clair : elles sont chiffrées avant enregistrement, puis déchiffrées uniquement au moment de l'appel au modèle.

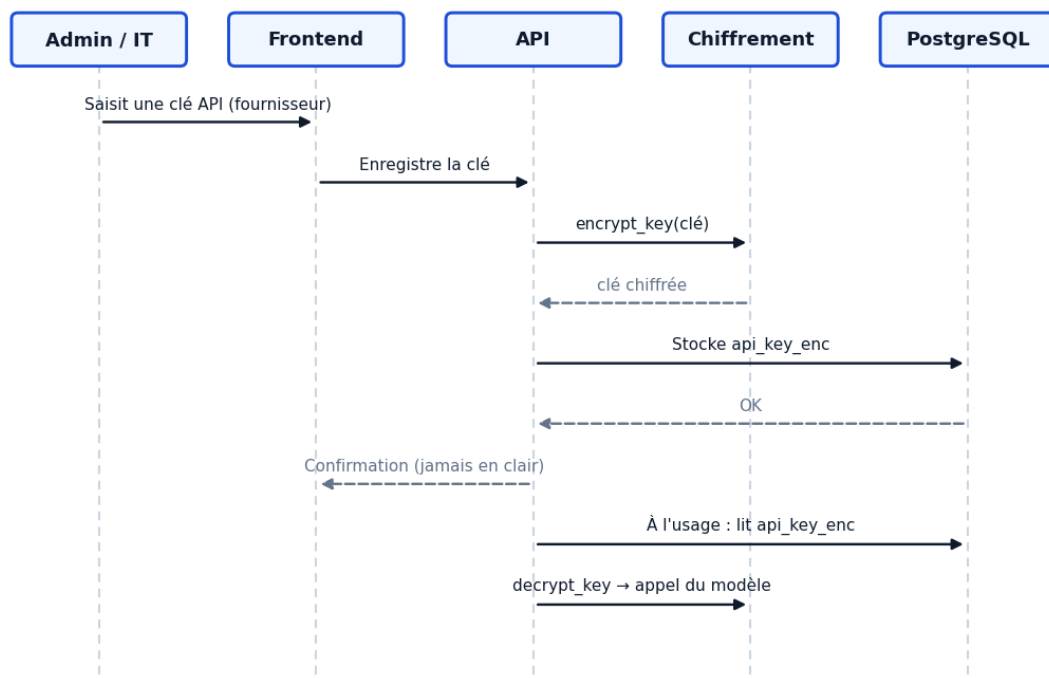


Diagramme de séquence — Gestion des clés API

7. Le pipeline RAG (cœur du projet)

Chaque document parcourt un pipeline complet, dont chaque étape est visualisable et paramétrable par l'ingénieur IT :

Étape	Description
1. Ingestion	Import de fichiers, Google Drive, scraping d'URL, crawl de site, sitemap, flux RSS.
2. Chargement	Extraction du texte brut et de la structure (Docling pour les PDF/DOCX/PPTX).
3. Nettoyage	Correction OCR, dé-duplication, masquage des données personnelles (PII).
4. Parsing	Structuration en sections, tableaux et images ; résumé des images pour les rendre indexables.
5. Découpage	Stratégies de chunking adaptées à chaque format (récursif, par page, par ligne, sémantique, hiérarchique).
6. Vectorisation	Calcul des embeddings (BGE-M3 local par défaut, ou fournisseur externe).
7. Indexation	Stockage des vecteurs dans pgvector.
8. Recherche	Recherche hybride (sémantique + mots-clés) avec pondération réglable.
9. Génération	Réponse produite par le LLM configuré, avec citation des sources et prompt système personnalisable.

L'ingénieur peut prévisualiser le résultat de chaque étape (texte chargé, contenu parsé, chunks générés) et même éditer manuellement le contenu extrait avant l'indexation.

8. Travail réalisé

Cette section présente, captures à l'appui, les étapes clés effectivement réalisées de la chaîne de traitement.

Étape 1 : Extraction de documents

Le document importé est d'abord chargé (texte brut), puis parsé en une structure riche : titres, sections, tableaux et images. Le contenu extrait est prévisualisable et éditable avant l'indexation.



Étape 1 — Extraction : texte chargé du document

CAPTURE D'ÉCRAN À INSÉRER

Étape 1 — Document parsé (sections, tableaux, images)

Étape 1 — Document parsé (sections, tableaux, images)

Étape 2 : Chunking (découpage)

Le contenu est découpé en fragments (chunks) selon une stratégie adaptée au format du document, avec des paramètres réglables (taille, chevauchement, stratégie par format).

CAPTURE D'ÉCRAN À INSÉRER

Étape 2 — Page de configuration du découpage (chunking)

Étape 2 — Page de configuration du découpage

Étape 3 : Embedding (vectorisation)

Chaque fragment est transformé en vecteur numérique par un modèle d'embedding (BGE-M3 local par défaut, ou un fournisseur externe).

CAPTURE D'ÉCRAN À INSÉRER

Étape 3 — Page de configuration de l'embedding

Étape 3 — Page de configuration de l'embedding

Étape 4 : Stockage vectoriel

Les vecteurs et leurs métadonnées sont stockés dans PostgreSQL via l'extension pgvector, avec un index dédié permettant une recherche par similarité efficace.

Étape 5 : Recherche hybride

À l'interrogation, le système combine une recherche sémantique (vecteurs) et une recherche par mots-clés, avec une pondération réglable, afin de sélectionner les passages les plus pertinents (top-k).

Étape 6 : Génération LLM

Les passages retrouvés sont fournis comme contexte au modèle de langage choisi, qui rédige une réponse en citant ses sources. Le fournisseur, le modèle et le prompt système sont configurables.

CAPTURE D'ÉCRAN À INSÉRER

Étape 6 — Choix du modèle LLM

Étape 6 — Choix du modèle LLM

9. Fonctionnalités réalisées

Gestion des accès et des rôles

Organisations, départements et utilisateurs avec les rôles Admin / IT / End-user ; relation plusieurs-à-plusieurs entre utilisateurs et départements ; authentification et invitations.

Espaces RAG entièrement configurables

Création d'un espace rattaché à un département, avec le réglage fin de toutes les étapes du pipeline. Chaque IT peut utiliser les fournisseurs de l'entreprise ou ses propres clés (chiffrées au repos).

Versioning des configurations et cycle de déploiement

C'est l'un des apports majeurs de la dernière phase :

- L'IT peut sauvegarder plusieurs versions d'une configuration (v1, v2...), chacune capturant l'ensemble des paramètres du pipeline ;
- il peut recharger une version dans l'éditeur pour la comparer ;
- il choisit ensuite la version à déployer auprès des utilisateurs finaux.

Un cycle de vie clair encadre l'espace : Brouillon -> Déployé -> En édition. L'IT peut « mettre en pause » un agent déployé pour ajouter des documents ou modifier la configuration, puis le re-déployer ; pendant ce temps, l'utilisateur final voit l'agent avec la mention « en mise à jour ».

Détection intelligente de ré-indexation

Le système distingue les paramètres qui impactent réellement l'index (découpage, embedding) de ceux qui n'agissent qu'au moment de la requête (LLM, température, prompt). Une ré-indexation n'est demandée que lorsque la configuration d'indexation change effectivement — l'ajout d'un simple document n'indexe que ce document, sans reconstruire tout l'index.

Modèle de permissions à deux niveaux

- Propriétaire : l'IT créateur de l'espace. Seul à pouvoir déployer, publier, importer et supprimer des documents.
- Collaborateurs IT : co-constructeurs invités par le propriétaire ; ils peuvent régler la configuration, créer des versions et tester, mais pas déployer ni gérer les documents.
- Espace privé : par défaut, un espace n'est visible que de son propriétaire et de son équipe IT — aucun utilisateur final n'y accède tant qu'il n'est pas explicitement publié.

Expérience utilisateur final

Interface repensée : liste des agents déployés, conversation en langage naturel avec citation des sources, et panneau « Documents » permettant d'ouvrir et de consulter les PDF originaux en ligne (visionneuse intégrée).

Prévisualisation IT des agents déployés

Une vue dédiée permet à l'IT de tester un agent déployé exactement comme le voit l'utilisateur final (même chat, mêmes documents), via un menu « RAG Spaces » à deux entrées : Espace de travail et Agents déployés.

10. Synthèse de l'avancement

Module	État	Avanc.
Gestion des utilisateurs / départements / rôles	Terminé	100%
Ingestion multi-source et multi-format	Terminé	100%
Chargement / parsing / nettoyage des documents	Terminé	95%
Découpage (chunking) par format	Terminé	95%
Embeddings et indexation (pgvector)	Terminé	100%
Recherche hybride et génération (LLM factory)	Terminé	95%
Versioning et cycle de déploiement	Terminé	90%

Module	État	Avanc.
Permissions (propriétaire / équipe / accès)	Terminé	90%
Expérience utilisateur final (chat + documents)	Terminé	85%
Module d'évaluation (comparaison des versions)	Planifié	10%
Auto-tuning / découpage agentique	Planifié	0%
Workflows, API Agent, Data Agent	En cours	30%

Avancement global estimé : environ 80 %. Le cœur fonctionnel (pipeline RAG complet, configuration, versioning, déploiement, permissions et consommation par l'utilisateur final) est opérationnel et testé de bout en bout.

11. Difficultés rencontrées et solutions

- Cohérence configuration / index. Un changement de découpage ou d'embedding rend l'index existant obsolète. Solution : un indicateur de ré-indexation calculé par empreinte des seuls paramètres d'indexation, pour n'alerter qu'en cas de changement réel.
- Permissions multi-couches. Concilier propriétaire, collaborateurs IT et accès des utilisateurs finaux sur un même point d'entrée de listing. Solution : un filtrage par rôle centralisé dans la couche service.
- Migrations de schéma sans Alembic. Solution : des migrations légères et idempotentes exécutées au démarrage (ADD COLUMN IF NOT EXISTS, conversion de types).
- Consultation de fichiers protégés en cross-origin. Solution : récupération du fichier en blob authentifié côté client avant affichage dans la visionneuse.

12. Prochaines étapes

1. Module d'évaluation : jeu de test et métriques (taux de réponse pertinente, MRR) pour comparer objectivement les versions et guider le choix de déploiement.
2. Auto-tuning : sélection automatique de la stratégie de découpage la mieux adaptée à chaque document.
3. Workflows et agents avancés (API Agent, Data Agent).
4. Historique de conversation côté utilisateur final.
5. Tests, sécurité et préparation au déploiement en production.

Le calendrier prévisionnel se décline en trois horizons :

Échéance	Objectif principal
Court terme	Module d'évaluation : métriques et comparaison objective des versions d'un espace.
Moyen terme	Auto-tuning du découpage ; workflows et agents avancés (API Agent, Data Agent).
Long terme	Tests approfondis, sécurité et mise en production.

13. Conclusion

Le projet AgentFlow a atteint un stade avancé et fonctionnel : l'ensemble de la chaîne RAG — de l'ingestion des documents jusqu'à la réponse citée à l'utilisateur final — est en place, avec une gestion complète des configurations, des versions, du déploiement et des permissions. Les prochaines étapes se concentreront sur l'évaluation quantitative des configurations et sur les fonctionnalités avancées, afin de consolider la qualité et de préparer une mise en production. Je reste à votre disposition pour en discuter plus en détail lors de la réunion.